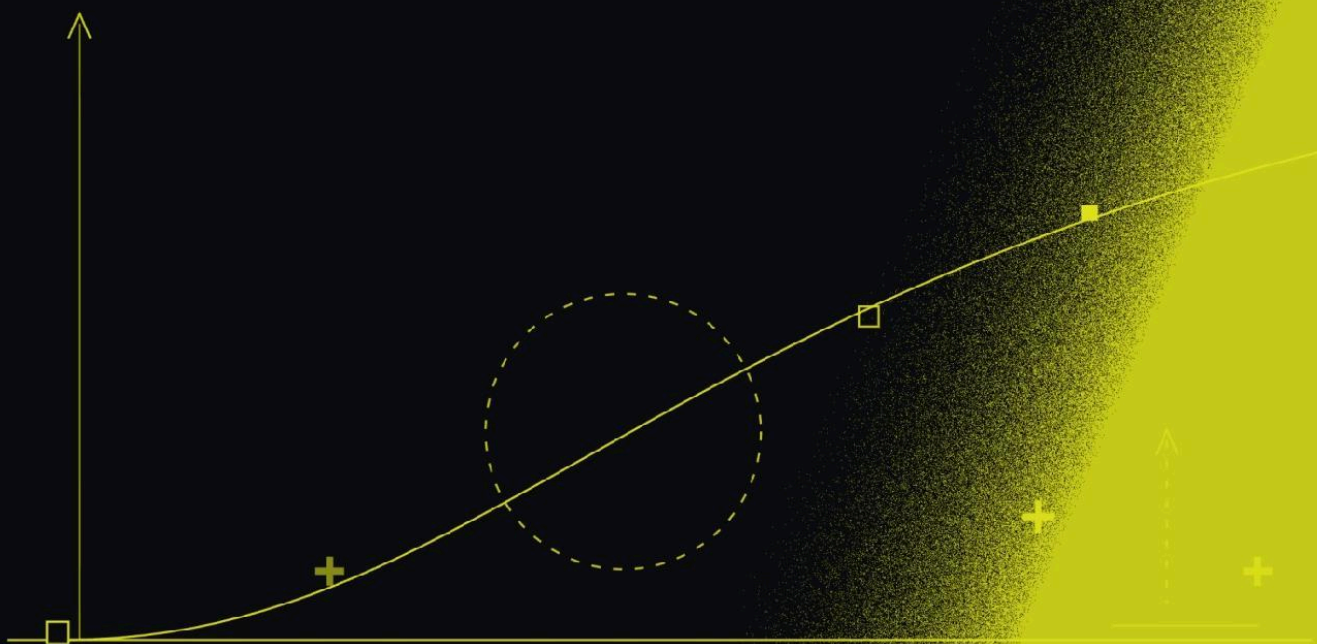


THE PHYSICAL AI SPRINT

A hackathon alongside Actuate SF



HOSTED BY

NEBIUS

 NVIDIA

 Antioch

 Toloka

Contents

THE EVENT

01	The challenge	3
02	Format	3
03	Schedule	4
04	What you get	4
05	Getting started guides	5
06	Judging	6
07	Ground rules	6

THE GUIDES

G1	LeRobot on macOS: first-time setup for the SO-101	8
G2	Sim teleoperation: drive the Antioch sim with a real SO-101	13
G3	Train your first SO-101 policy: ACT/Diffusion on sim data	18
G4	Running Unitree robots on Antioch: Isaac Sim vs Isaac Lab	21

01

The challenge

Modern AI excels at generating text and pixels. Physical AI requires closing the loop between intelligence and embodiment across three pillars:

Perception

Interpret camera and sensor data into a live understanding of a scene.

Reasoning

Turn goals into multi-step behavior and adapt when the world changes.

Action

Translate plans into robot motion that actually works.

Your goal: design and demonstrate a Physical AI task in simulation, in the real world, or across both. You have several robot modalities, the Antioch simulation platform for building and testing, and a single day! The brief is deliberately open-ended: we provide the robots, the world, and the building blocks, and you decide what the system does and how it thinks. In short, build some cool stuff.

There are three broad project directions. All are equally viable:

- **Simulation only.** Build and demonstrate entirely on the Antioch platform. No hardware queue, ready-made workflows, and libraries of digital twins and assets.
- **Hardware only.** Build directly on the physical robots at the hardware stations, or **use your own robot.**
- **Sim and real.** Show a trajectory between the two: prototype or train in sim and run it on the real robot (sim-to-real), or capture real demonstrations and bring them into sim (real-to-sim).

Hardware is limited and shared across all teams, so simulation is the dependable path. Many of the strongest projects will likely run most of their loop in sim even when they touch hardware.

02

Format

100 to 125 participants, hacking in teams of 4 or 5.

03

Schedule

8:00 am	Doors open and check-in, grab breakfast
9:00 am	Welcome and challenge briefing
9:15 am	Technical workshops by hosts
10:30 am	Hacking begins

12:00 pm	Lunch
1:00 pm	Hacking continues
3:30 pm	Submission deadline
4:30 pm	Demos and judging (top 6 live demos)
5:30 pm	Winners announced

04

What you get

The Antioch platform

Antioch is a cloud simulation platform for robotics development. You write Python against the NVIDIA Isaac stack (Isaac Sim and Isaac Lab are both available) on your own laptop, and Antioch dispatches it to GPU machines on Nebius. Nothing heavy runs locally: no GPU, no Docker, no Isaac install, and macOS and Linux both work.

You can develop interactively by building simulations and streaming them into the browser. You can also define parameterized simulation scenarios and run variants at scale headlessly. Scenario runs are recorded and replayable, with metrics, logs, live viewport streams, and 3D recordings you can open in the browser.

Your team will receive a link to sign up for the Antioch Console. Each team gets its own development environment where teammates share machines and scenario runs and can see each other's work as it lands. Signup links will be distributed once teams are formed.

This section is brief on purpose. To understand how the platform actually works, start with the Antioch product docs (accessible after sign-up for the Antioch platform with your team):

<https://console.preview.antioch.com/docs/welcome>

Robot hardware

- **10x LeRobot SO-101 leader/follower arm pairs**
 - [Setup documentation](#) for Linux and Windows
 - For macOS, use Guide 1 in this packet
- **2x Unitree Go2-W wheeled quadrupeds**
 - [Documentation](#)
- **1x Unitree G1 humanoid**
 - [Documentation](#)
- **Your own robot**
 - Work on hardware only, or bring an open-source digital twin into the Antioch platform.

Hardware stations are shared between teams and staffed by the host teams.

Sim-ready assets

- **Robots:** a calibrated SO-101 digital twin, plus the Unitree lineup in the Isaac asset library (Go2-W, G1, and others; Guide 4 lists what is on the shelf).
 - **NOTE:** To access Unitree robot digital twins in Antioch, you must be using Isaac Lab (not Isaac Sim)
- **Environments:** ready-made scenes to stage your task in.
- **Objects:** containers, fixtures, and props, so composing a scene takes minutes rather than hours of importing and tuning your own meshes.

Physical assets for SO-101

- Blocks of various size, shape, and color
- Vials and vial racks

Support

Engineers from the host companies are on hand all day for platform, hardware, and training questions.

05

Getting started guides

The four guides at the back of this packet each take you from zero to a working proof of concept with the hardware or with Antioch:

- **Guide 1. LeRobot on macOS:** first-time setup for the SO-101. From a Mac with nothing installed to a real arm following your hand.
- **Guide 2. Sim teleoperation:** drive the Antioch sim with a real SO-101 leader arm and record scored demonstration episodes.
- **Guide 3. Policy training:** train an ACT or Diffusion policy on sim data, evaluate it closed-loop, and run it on a real arm.
- **Guide 4. Unitree robots on Antioch:** spawn a quadruped or humanoid, give it a controller, and verify it on physics rather than video.

These exist so you can explore the stack and prove the plumbing works. They are not the assignment. The challenge is what you build past them: extend one into something of your own, combine several, or ignore them entirely and chart a different course.

Some directions to calibrate ambition: a vial pick-and-place cell that survives a shuffled scene, a new skill taught by teleop and distilled into a policy, person-following with a quadruped, pouring with the G1, a multi-robot scene, natural-language tasking, deliberate failure recovery.

06

Judging

Judging will have two rounds.

Round 1: Science-fair style. Judges from the hosting companies circulate and view projects at team stations, so have a tight walkthrough and a live or recorded demo ready. 6 teams are selected as finalists.

Round 2: The top 6 teams then demo to the full group, and winners are announced after.

Projects are judged on four categories, equally weighted. The judging rubric is the same for both rounds.:

CATEGORY	WHAT JUDGES LOOK FOR
Ambition	How hard the problem is and how much of the loop you take on.
Functionality	Does it work end-to-end when demonstrated.
Creativity	Originality in the task, approach, or demo.
Architectural quality	How well the system is put together: clean boundaries between perception, reasoning, and action, and sound engineering choices.

07

Submission Format

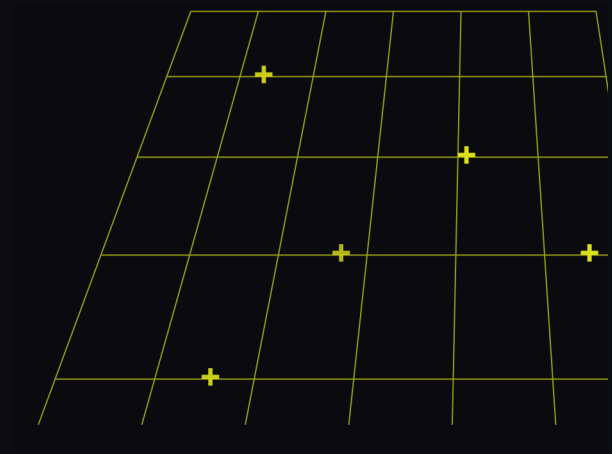
You may show your project in whatever format suits you. We suggest having a demo ready to show and/or a walkthrough. Slides, code walkthrough, etc. are also options.

The 6 finalist teams will present on stage and have access to a projector.

08

Ground rules

- Unless you are confident you can pull it off quickly, avoid fine-tuning pre-trained policies or VLA models. Training a small imitation policy from scratch on your own demonstrations fits comfortably in the day (Guide 3 shows the path); wrangling a large pre-trained model usually does not.
- **Make it awesome.**



GETTING STARTED

The Guides

Four walkthroughs, each from zero to a working proof of concept. Prove the plumbing works, then build past them.

1 LeRobot on macOS

First-time setup for the SO-101 — from a bare Mac to a real arm following your hand.

2 Sim teleoperation

Drive the Antioch sim with a real leader arm and record scored demonstration episodes.

3 Policy training

Train ACT or Diffusion on sim data, evaluate closed-loop, run it on a real arm.

4 Unitree robots on Antioch

Spawn a quadruped or humanoid, give it a controller, verify it on physics.

GUIDE 1

LeRobot on macOS: first-time setup for the SO-101

Go from a Mac with nothing installed to a real SO-101 arm following your hand. The published LeRobot instructions assume Windows or Ubuntu, so the port names, permissions, and GPU advice in them do not match a Mac. This guide is the macOS version, with every difference called out where you hit it.

TOTAL TIME	About 45 minutes, most of it calibration.
TESTED AGAINST	lerobot 0.4.4, Python 3.10, macOS on Apple Silicon.
VENDOR MANUAL	Assembly and servo wiring: SO-ARM101 user manual (docs.hiwonder.com)

Step 0: What you need

Hardware on the desk:

- An SO-101 **follower arm**, which is the one that moves under command.
- An SO-101 **leader arm** if you want to teleoperate. The leader has a handle and no gripper servo horn.
- One **data-capable USB-C cable per arm**, plus the arm power supplies. A charge-only cable shows no serial port at all and wastes twenty minutes.

Software: a terminal, and either miniconda or uv (docs.astral.sh/uv). You do not need a GPU, Docker, or ROS.

Step 1: Create the environment

Either tool works. The conda route pulls in ffmpeg, which LeRobot uses for video encoding during dataset recording:

```
conda create -n lerobot python=3.10 ffmpeg -c conda-forge -y
conda activate lerobot
pip install lerobot opencv-python
```

With uv instead:

```
uv venv ~/lerobot-env --python 3.10
source ~/lerobot-env/bin/activate
uv pip install lerobot opencv-python
```

Add `opencv-python` in either case, because anything that touches a camera needs it and the error you get without it is unhelpful.

CHECKPOINT

```
python -c "import lerobot, torch; print(lerobot.__version__, torch.__version__)"
python -c "import torch; print('MPS available:', torch.backends.mps.is_available())"
```

You should see a lerobot version and MPS available: True. Plain `pip install torch` already gives you Apple Silicon acceleration. There is no CUDA wheel for a Mac, so ignore every tutorial that tells you to install one.

Step 2: Find the serial ports

Plug in one arm, then list the ports:

```
ls /dev/tty.usbmodem*
```

macOS names serial ports `/dev/tty.usbmodemXXXXXXXX`. Linux tutorials say `/dev/ttyACM0` and the vendor manual says COM22 or COM24; neither exists here, so substitute your own port name into every command you copy. LeRobot also ships an interactive finder that walks you through unplugging a cable:

```
lerobot-find-port
```

Two more macOS specifics. You do not need `udev` rules or `sudo chmod` on the port, because it is usable as soon as it appears. USB hubs also renumber ports across reboots and replugs, so check the name at the start of each session rather than trusting one you wrote down yesterday.

CHECKPOINT

`ls /dev/tty.usbmodem*` prints one port per plugged-in arm. If it prints nothing, swap the cable before you debug anything else.

Step 3: Tell the two arms apart

Both arms produce identical port names, so you have to identify them. Two reliable methods:

- **The unplug test.** List the ports with both arms connected, unplug one, list again, and note which name disappeared.
- **The passive wiggle test.** This project ships a script that reads joints for 15 seconds and commands no motion:

```
cd ~/my-sim
python replay/check_so101.py --port /dev/tty.usbmodemXXXXXXXX
```

Move each joint gently by hand while it prints. The sanity bar comes from the script's own docstring: at rest the numbers should be repeatable, none should sit near plus or minus 180, and moving one joint must move only its own number, in a consistent direction.

CHECKPOINT

You know which port is the leader and which is the follower. Save them so later commands are copy-paste:

```
export LEADER=/dev/tty.usbmodemAAAAAAA
export FOLLOWER=/dev/tty.usbmodemBBBBBBB
```

One expected surprise: on most lerobot versions `connect()` enables torque, so the arm stiffens when the script starts. That is normal, and the script still sends no motion.

Step 4: Assign servo IDs (DIY kits only)

Pre-assembled arms arrive with servo IDs set, so skip this step. If you built the arm from a kit, each servo needs its ID written one at a time, with only that servo connected:

```
lerobot-setup-motors --robot.type=so101_follower --robot.port=$FOLLOWER
```

Follow the prompts and the vendor manual's wiring order. Connecting several unconfigured servos at once writes the same ID to more than one of them, which is confusing to undo.

Step 5: Calibrate each arm

Calibration teaches LeRobot each joint's real range. Do it once per arm, in the calibration start pose from the manual:

```
lerobot-calibrate --robot.type=so101_follower \  
  --robot.port=$FOLLOWER --robot.id=my_follower_arm \  
lerobot-calibrate --teleop.type=so101_leader \  
  --teleop.port=$LEADER --teleop.id=my_leader_arm
```

The `--id` you choose here is the `--id` every later command must use, because profiles are keyed by it. They are written to `~/.cache/huggingface/lerobot/calibration/`.

CHECKPOINT

Both profiles exist:

```
ls ~/.cache/huggingface/lerobot/calibration/
```

Back that directory up now. Recalibrating in the middle of a demo costs you the demo.

Step 6: First motion: teleoperate

With both arms calibrated, the leader drives the follower:

```
lerobot-teleoperate \  
  --robot.type=so101_follower --robot.port=$FOLLOWER --robot.id=my_follower_arm \  
  --teleop.type=so101_leader --teleop.port=$LEADER --teleop.id=my_leader_arm
```

Keep one hand near the power switch for the first run. Move the leader slowly and watch for a joint that travels the wrong way, which points at a calibration or ID mismatch rather than a hardware fault.

CHECKPOINT

The follower mirrors the leader across all six joints, including the gripper, with no joint racing to its limit. Flags vary slightly between lerobot releases, so run `lerobot-teleoperate --help` if one is rejected.

Step 7: Record a dataset

Recording captures synchronized camera frames, joint states, and commands in the LeRobot dataset format, which is what the training scripts consume. Check your cameras first:

```
lerobot-find-cameras opencv
```

macOS gates camera access per application, so grant your terminal permission under System Settings → Privacy and Security → Camera. Without it the capture returns black frames and reports no error at all.

Then record, adding your camera and dataset flags from `lerobot-record --help`:

```
lerobot-record \  
  --robot.type=so101_follower --robot.port=$FOLLOWER --robot.id=my_follower_arm \  
  --teleop.type=so101_leader --teleop.port=$LEADER --teleop.id=my_leader_arm \  
  --dataset.repo_id=<your-name>/so101-pick-place \  
  --dataset.repo_id=<your-name>/so101-pick-place
```

```
--dataset.single_task="Put the green cube in the tray."
```

CHECKPOINT

Episodes appear under `~/cache/huggingface/lerobot/<your-repo-id>/` with video files and parquet data. From there, `lerobot-train` fits a policy; Guide 3 covers the training and evaluation path, including a way to generate episodes in simulation instead of by hand.

macOS differences at a glance

EVERY OTHER TUTORIAL SAYS

ON A MAC

`/dev/ttyACM0` or **COM22**

`/dev/tty.usbmodemXXXXXXX`

`sudo chmod 666 /dev/ttyACM0` and **udev rules**

Nothing to do; the port works when it appears

Install a CUDA build of torch

`pip install torch` gives MPS; no CUDA wheel exists

Cameras work once plugged in

Grant terminal camera access in Privacy and Security

Port names are stable

Hubs renumber ports on reboot and replug

Troubleshooting

SYMPTOM

CAUSE AND FIX

`ls /dev/tty.usbmodem*` **prints nothing**

Charge-only cable, or the plug is not seated. Swap to a data cable.

Both arms show the same kind of port name

Expected. Identify them by the unplug test or `check_so101.py` (Step 3).

A calibration or teleop command cannot find the profile

The `--id` does not match the one used at calibration. Check `~/cache/huggingface/lerobot/calibration/`.

One joint moves the wrong direction

Calibration or servo ID mismatch on that joint. Recalibrate that arm; check IDs on a DIY kit.

Camera capture returns black frames

macOS camera permission for your terminal (Step 7).

The arm stiffens as soon as a script connects

Expected: `connect()` enables torque. Passive scripts still send no motion.

Port worked yesterday, not today

The hub renumbered it. Re-run `ls /dev/tty.usbmodem*` and update `$LEADER` and `$FOLLOWER`.

A script refuses to run over a version or units stamp

Do not bypass it. Re-stamp in the environment you are running from.

House rules for real hardware

These are enforced by the scripts in `replay/`, and they exist because each one was learned the expensive way.

- **Pin one driver per data file.** Every trajectory and policy records the driver, `lerobot` version, and calibration id it was made with, and the loader validates all three before anything moves. A mismatch stops the run loudly instead of falling back.
- **Treat units as data.** The units dictionary travels with the file and is asserted against the robot config at both ends.

- **Never cut torque while the arm holds a commanded pose**, because the arm drops. Ramp back to the rest pose read at connect time, then disconnect.
 - **Run the real `--dry-run` first**, every time the code or the trajectory changed. It exercises the whole path with sends suppressed. Note that `--dry-lift-mm` is a rehearsal that does move the arm.
 - **Keep a hand near the power switch** on first runs.
-

Scripts referenced here: `replay/check_so101.py`, `replay/replay_on_so101.py`, `replay/run_act_on_so101.py`. Sim teleop and cloud setup: Guide 2.

GUIDE 2

Sim teleoperation: drive the Antioch sim with a real SO-101 leader arm

Hold a real leader arm, watch a physics-simulated SO-101 mirror your hand on a cloud GPU, and record demonstration episodes that are scored the moment you stop recording. Nothing runs locally except the arm driver, so your Mac needs no GPU, no Docker, and no Isaac install.

FIRST MOTION About 5 minutes once the setup in Step 0 is done.

TESTED AGAINST antioch-sim 0.3.27 on isaac-601-ga, lerobot 0.4.4, Python 3.10 in the robot env, macOS on Apple Silicon.



→ scored demo episodes (mp4 + npz artifacts)

→ WebRTC livestream (your browser)

Step 0: One-time setup

You need three things before the fun part.

1. The project at `~/my-sim`, with its own environment:

```
cd ~/my-sim && uv sync && .venv/bin/antioch auth login
```

2. A separate environment that talks to the arm over USB. Keep it separate from the project environment, because mixing the two causes the most confusing errors in this whole pipeline:

```
conda create -n lerobot python=3.10 ffmpeg -c conda-forge -y
conda activate lerobot
pip install lerobot opencv-python
```

3. A calibrated leader arm. If yours has never been calibrated, follow Guide 1 through Step 5 and come back. The follower arm stays in its box for this guide, because the sim replaces it.

ONE RULE FOR EVERY COMMAND BELOW

Call the project's own CLI at `~/my-sim/.venv/bin/antioch` from inside `~/my-sim`. A bare `antioch` on your `PATH` is a different binary that cannot see the project, and the CLI refuses to run outside a directory holding `antioch.yaml`.

Step 1: Two terminals

This pipeline needs two terminals at once, and it helps to know which is which before you start.

- **Terminal A is the sim session**, run from `~/my-sim` with the project environment. It claims the cloud machine and keeps the simulation alive.
- **Terminal B is the bridge**, run in your `(lerobot)` environment. It reads the arm and streams joints into the machine that Terminal A claimed.

Step 2: Find the leader port

Plug in the leader arm by itself, then capture the port in one command:

```
export LEADER=$(ls /dev/tty.usbmodem* | head -1) && echo $LEADER
```

CHECKPOINT

It echoes something like `/dev/tty.usbmodem58FA0836661`. If it prints nothing, the cable is charge-only or unseated. If you have both arms plugged in, this picks one arbitrarily, so unplug the follower or identify the ports as described in Guide 1, Step 3.

Step 3: Warm a machine and start the session

In Terminal A. The first command claims a cloud machine and warms it, which matters because a freshly claimed machine deadlocks on its first render and a teleop session started cold dies a few minutes in with no useful error:

Then start a 30 minute session on that same machine. Leave off `--machine` entirely and the session uses the machine you just warmed:

CHECKPOINT

The output prints `[teleop] live for 1800s; waiting for /tmp/leader_stream.jsonl`. The simulation is up and idling. Leave this terminal alone for the rest of the session.

Step 4: Open the livestream

In a third shell, or Terminal B before you connect the arm, open the machine's viewport straight into your browser:

CHECKPOINT

A browser tab shows the arm, a green cube, and a blue tray. The stream starts when a viewer connects, so leave the tab open. Every new session on a new machine has a new URL, so re-run this command rather than reusing an old tab.

Step 5: Connect the arm

In Terminal B, in your `(lerobot)` environment. One command, no machine id to look up, because the wrapper resolves the assigned machine itself:

Example snippet

```
# SO-101 live teleop on Antioch: move the real leader arm, the cloud sim mirrors it.
#
# leader arm (USB) -> laptop bridge -> localhost:56321 -> Antioch tunnel
# -> scenario listener on the GPU machine -> sim follower -> livestream back
#
# antioch.yaml: services.sim.ports: [{name: teleop, target: 56321, published: 56321}]
# run: antioch services up | antioch scenario run --scenario so101_live_teleop --stream

# ---- laptop half: read the leader, stream JSON lines at 30 Hz -----
leader = SO101Leader(SO101LeaderConfig(port="/dev/tty.usbmodemXXX", use_degrees=True))
leader.connect()
sock = socket.create_connection(("127.0.0.1", 56321)) # rides the tunnel
sock.sendall(b'{"header": true, "units": {"joints": "degrees", "gripper": "0_100"}}\n')
assert json.loads(sock.makefile().readline())["ack"] # scenario is live
while True:
```

```

sock.sendall((json.dumps(leader.get_action()) + "\n").encode())
time.sleep(1 / 30)

# ---- cloud half: the Antioch scenario listens and mirrors -----
@antioch.scenario(sim=antioch.BootProfile(physics_dt=0.005, render_dt=0.02))
def so101_live_teleop(run, listen_port: int = 56321, max_seconds: float = 600.0):
    antioch.load_asset("so101_antioch", prim_path="/World/SO101", version="1.3.2")
    world, robot = antioch.world(), Articulation(root) # after world.reset()
    server.bind(("0.0.0.0", listen_port)); server.listen(1)
    while sim_time < max_seconds:
        newest = drain_socket_keep_newest(conn) # never replay backlog
        if newest:
            # degrees -> radians,
            targets[arm_idx] = clip(radians(newest), lo, hi) # gripper 0..100 -> Jaw range
            robot.set_dof_position_targets(targets) # mirror this tick
            world.step(render=True) # 50 Hz + livestream

```

Step 6: Record episodes

Click into Terminal B first, because the bridge reads single keypresses and keys pressed in the browser do nothing.

The collection rhythm:

```

r → new cube position
s → recording starts
   pick the cube up with the leader, place it in the tray
s → episode saved and auto-scored
r → again

```

KEY	EFFECT
r	Reset: the cube teleports to a random spot on the table
s	Start recording, and press again to stop and save
Ctrl-C	Stop the bridge; the sim session keeps running

CHECKPOINT

After each stop, Terminal A prints episode N closed: success/fail, NNN frames. Success means the cube was lifted and ended inside the tray, judged by the same gates the automated test suites use, so it is a real score rather than a self-report.

Aim for quality over quantity: smooth approach, clean grasp, deliberate place. Ten to fifteen good episodes per session is a solid haul. Episodes upload as they close, so a session that dies loses at most the episode in progress.

Step 7: Download your data

From ~/my-sim, once you are done. This resolves the newest run id for you:

```

.venv/bin/antioch scenario download $(.venv/bin/antioch scenario list --limit 1 \
| grep -oE '[0-9a-f]{32}' | head -1)

```

CHECKPOINT

You get `epNNN_success.mp4`, the robot's wrist camera view at 640×480 and 10 Hz, paired with `epNNN_success.npz` holding joint states and your leader commands aligned at 10 Hz. That pair is exactly what the training pipeline ingests, so Guide 3 picks up from here.

Following the LeRobot convention, the actions in the `.npz` are your commanded leader targets rather than the achieved joint positions. Train on it as it is, because a policy trained on achieved positions cannot hold a load.

Why the pipe uses machine exec

This is the one detail worth understanding, because getting it wrong fails silently.

`antioch machine exec` forwards `stdin` into the container where the scenario runs. `antioch machine ssh` lands you in a separate ephemeral filesystem, so a `/tmp/leader_stream.jsonl` written there is invisible to the scenario.

The sim arm then never moves, with no error printed anywhere. The wrapper script uses `machine exec` for exactly this reason, which is why the recommended path is to run the wrapper rather than assemble the pipe by hand.

Troubleshooting

Two things to avoid. Do not edit a scenario mid-session and expect it to take effect, because the container keeps the code the session launched with; cycle the session instead. Do not `kill` inside the container, because that can wedge the `exec` channel and take the bridge down with it. Let sessions expire on `max_seconds`, or use `antioch machine reset` and then warm the machine again.

SYMPTOM	CAUSE AND FIX
<code>BrokenPipeError</code> from the bridge	The pipe went through <code>machine ssh</code> instead of <code>machine exec</code> . Use <code>teleop_pipe.sh</code> .
no machine assigned from the wrapper	Terminal A is not running. Start the session first, because the wrapper only connects and never claims.
Assigning machine... waiting spinner	A stale machine id made the CLI claim a new machine. Stop it and let the wrapper resolve the id itself.
No <code>antioch.yaml</code> found	You ran an <code>antioch</code> command outside <code>~/my-sim</code> . <code>cd ~/my-sim</code> first.
Session hangs at start and dies a few minutes in	The machine was cold. Run <code>so101_render_probe</code> on it, then relaunch the session.
Stream is fine but the sim arm ignores you	Check Terminal A for bridge header <code>ok</code> and a rising frame count. If neither appears, restart the bridge.
Pressed <code>s</code> and nothing recorded	The keypress went to the browser. Click Terminal B, then watch for the <code># record ON echo</code> .
Stream URL is dead	The platform rotated machines, so the id and URL both changed. Restart from Step 3.
<code>ls /dev/tty.usbmodem*</code> shows nothing	Charge-only cable or a loose plug. Use a data-capable USB-C cable.

GUIDE 3

Train your first SO-101 policy: ACT/Diffusion on sim data

Generate demonstration episodes in simulation, train an imitation policy on them, check it closed-loop in sim, and run it on a real arm. You need the my-sim project with Antioch access, plus a Python 3.10 env with lerobot, torch, and opencv-python. Your laptop needs no GPU and no Isaac install: simulation runs on cloud GPUs, and training runs locally on Apple Silicon (MPS).

FIRST POLICY About 1.5 hours, most of it unattended training.

TESTED AGAINST lerobot 0.4.4, torch 2.10 for training and 2.11+cu130 for remote eval, Python 3.10 in the lerobot env, antioch-sim 0.3.27 on isaac-601-ga, asset so101_antioch 1.3.0.

Step 1: Generate episodes

From the project root, run the scripted-expert suite. It replays a verified pick-and-place over randomized cube positions and sizes:

```
.venv/bin/antioch scenario list --limit 5      # find the run ids
.venv/bin/antioch scenario download <run-id>
```

CHECKPOINT

About 30 episodes appear in roughly 4 minutes of suite time. Each episode is a pair of files: epNNN_success.npz with joint states and actions at 10 Hz, and epNNN_success.mp4 with the 640×480 wrist camera view.

If you would rather record human demonstrations, drive the sim arm with a real leader arm instead (Guide 2). The output format is the same; plan for 10 to 15 episodes per session.

IMPORTANT

Whichever route you take, the actions array must hold the commanded joint targets rather than the measured positions. A policy trained on measured positions learns to command whatever pose the arm already has, so it stops producing force the moment it meets resistance and freezes under load. Both routes already record commands; keep it that way if you modify them.

Step 2: Build the dataset

Convert the episode pairs into a LeRobot dataset with exactly these fields:

FIELD	VALUE
observation.images.wrist	640×480 RGB, 10 Hz
observation.state	6 joint positions, radians
action	6 next-step joint targets, radians
task string	"Put the green cube in the tray."

CHECKPOINT

About 30 episodes of 240 steps each load without errors. That is enough for a first policy. Add more data only after eval shows you need it.

Step 3: Train

Train ACT with the LeRobot default config (chunk 100), batch 8, inside your lerobot env:

```
lerobot-train --policy.type=act --dataset.repo_id=<your-dataset> \
--steps 15000 --batch_size 8 --save_freq 5000
```

Set the budget to 15k steps rather than the 100k in the LeRobot tutorial. The tutorial assumes large, messy, human-collected datasets; 30 consistent scripted episodes converge much earlier. For Diffusion Policy, swap in `--policy.type=diffusion` and expect a longer wall clock (about 90 M parameters against ACT's 52 M).

CHECKPOINT

Training runs at about 4 steps/s on MPS and finishes 15k steps in roughly an hour. The smoothed loss should end below about 20% of its value at step 500, with no NaN. Keep all three checkpoints; Step 4 decides which one is best.

Step 4: Evaluate closed-loop in sim

A low training loss tells you the policy can imitate the dataset. It does not tell you the policy can do the task. To find out, export the checkpoint as TorchScript (trace it on torch 2.10; the remote's 2.11 loads it) and run the eval scenario. The policy takes the scripted expert's place at 10 Hz: it receives the wrist image and the measured joints, and it outputs joint targets. It receives nothing else.

The run scores against the same physics gates as the expert suite: the cube must end inside the tray within the tick budget.

EVAL SET	EPISODES	PASS BAR
Seen positions (training bearings)	10	<i>TBD</i>
Held-out positions (midpoints)	9	<i>TBD</i>
Held-out sizes (47.5 / 52.5 mm)	6	reported

CHECKPOINT

A passing checkpoint clears both bars. If the seen set passes and held-out fails, add data diversity. If both fail, check the commanded-actions rule from Step 1 before you touch anything else.

Step 5: Run on the real arm

CHECKPOINT

`--dry-run` prints every command it would send while moving nothing. The live run performs the task. The script also enforces the safety rules: it validates the stamped driver, lerobot version, and calibration id at load, asserts units at both ends, and ramps torque back to rest before it disconnects. Leave those checks alone.

Troubleshooting

SYMPTOM

FIX

Policy grasps, carries, then freezes mid-task

It was trained on measured positions. Rebuild the dataset from commanded targets (Step 1).

Identical failures across all checkpoints

The first camera frame was black. Warm-render before the first decision, and reuse the previous frame when a capture returns nothing.

First render run on a fresh cloud machine hangs

The machine is cold. Warm it with `antioch scenario run --scenario so101_render_probe` (about 25 s), then rerun.

Loss looks great, eval fails

This is expected sometimes. The loss curve does not predict closed-loop success; only Step 4 does.

Training crawls or crashes on Mac

Use plain `pip install torch` for MPS. There is no CUDA wheel for Mac, and you do not need one.

Real arm ignores the policy file

The version stamp does not match. Re-stamp (`--stamp`) in the env you will run with, using the calibration id you calibrated with.

GUIDE 4

Running Unitree robots on Antioch: Isaac Sim vs Isaac Lab

Spawn a Unitree quadruped or humanoid, give it a controller, and prove it works. The first decision is which engine you run on, because Isaac Sim and Isaac Lab expose completely different Python entry points for the same robot. This guide covers both, then walks the path from a stock asset to a verified walking robot.

STANDING ROBOT About 15 minutes on an engine you have already chosen.

TESTED AGAINST antioch-sim 0.3.27, engines isaac-601-ga (Isaac Sim 6.0.1) and isaac-lab-30b2 (Isaac Lab 3.0), URDF importer 3.11.2, Unitree URDFs from unitreerobotics/unitree_ros, policies from fan-ziqi/rl_sar.

Step 1: Choose your engine

One project runs one engine, selected in three places that must agree: the antioch-sim extra in `pyproject.toml`, the image coordinate or Dockerfile in `antioch.yaml`, and the FROM line of that Dockerfile. Changing engines means editing all three and rebuilding the image.

The two engines are not two flavors of the same API. They are different libraries:

	ISAAC SIM 6.0.1 (ISAAC-601-GA)	ISAAC LAB 3.0 (ISAAC-LAB-30B2)
Simulation context	<code>isaacsim.core.api.World</code>	<code>isaacsim.sim.SimulationContext(SimulationCfg(...))</code>
antioch.world()	works	raises <code>UnsupportedEngineError</code>
Robot object	<code>isaacsim.core.api.robots.Robot</code> plus <code>add_reference_to_stage</code>	<code>isaacsim.assets.Articulation(ArticulationCfg(...))</code>
Prebuilt robot configs	none; you point at a USD	<code>isaacsim_assets.robots.unitree</code>
Gains	authored in the USD DriveAPI, or <code>set_gains</code> at runtime	declared in the <code>cfg</code> 's actuator groups
Commands	<code>controller.apply_action(ArticulationAction(...))</code>	<code>set_joint_position_target</code> then <code>write_data_to_sim</code>
Camera	<code>isaacsim.sensors.camera.Camera</code>	<code>isaacsim.sensors.Camera</code>
Physics manager	the PhysX default works	needs <code>isaacsim_ovphysx</code> , see the gotchas below

Choose Isaac Sim when you want a robot standing, driving, or running a pretrained policy, which is what this guide's verified runs use. Choose Isaac Lab when you want its `ArticulationCfg` library or its managers for training.

THE SAME 15 LINES ON EACH ENGINE — ISAAC SIM 6.0.1

```
from isaacsim.core.api import World
from isaacsim.core.api.robots import Robot
from isaacsim.core.utils.stage import add_reference_to_stage
from isaacsim.core.utils.types import ArticulationAction
```

```

world = World(physics_dt=1/200, rendering_dt=1/200, stage_units_in_meters=1.0)
world.scene.add_ground_plane(z_position=0.0)
add_reference_to_stage(usd_path=usd, prim_path="/World/robot")
robot = world.scene.add(Robot(prim_path="/World/robot", name="robot",
                             position=np.array([0.0, 0.0, 0.42])))
world.reset()
controller = robot.get_articulation_controller()
controller.set_gains(kps=kp, kds=kd)
controller.apply_action(ArticulationAction(joint_positions=targets))
world.step(render=True)

```

ISAAC LAB 3.0

```

import isaacsim as sim_utils
from isaacsim.actuators import ImplicitActuatorCfg
from isaacsim.assets import Articulation, ArticulationCfg
from isaacsim.sim import SimulationCfg, SimulationContext
from isaacsim_ovphysx.physics import OvPhysxCfg

sim = SimulationContext(SimulationCfg(dt=1/200, device="cuda:0",
                                     physics=OvPhysxCfg()))
robot = Articulation(ArticulationCfg(
    prim_path="/World/robot",
    spawn=sim_utils.UsdFileCfg(usd_path=usd),
    init_state=ArticulationCfg.InitialStateCfg(pos=(0.0, 0.0, 0.42)),
    actuators={"all": ImplicitActuatorCfg(joint_names_expr=[".*"],
                                           stiffness=25.0, damping=0.5)}))
sim.reset()
robot.set_joint_position_target(targets)
robot.write_data_to_sim()
sim.step(render=True)
robot.update(1/200)

```

Keep both Isaac imports inside the scenario function. The CLI imports your scenario files on your laptop, where neither library is installed.

Step 2: Check what already exists

Before building anything, find out whether your robot is already on the shelf. The Isaac 6.0 asset library carries these Unitree models:

```
A1 B2 Dex3 Dex5 G1 G1_23dof Go1 Go2 H1 H2 Z1 aliengo laikago
```

Isaac Lab additionally ships ArticulationCfg objects with tuned gains: UNITREE_A1_CFG, UNITREE_GO1_CFG, UNITREE_GO2_CFG, H1_CFG, G1_CFG, G1_MINIMAL_CFG, and G1_29DOF_CFG. Those configs live in isaacsim_assets, which is importable on the Lab engine only.

Nothing wheeled exists in either place. The Go2W, B2W, and similar variants are absent from the asset library and from isaacsim_assets, so they need Step 4.

To check for yourself rather than trusting a list, src/unitree_go2w_probe.py lists the library and the config module in one boot:

CHECKPOINT

The run prints the Unitree folder listing and whether the config module imported. On isaac-601-ga expect No module named 'isaacsim_assets', which is normal and tells you the gains must come from the USD or from your own table.

Step 3: Spawn a stock robot

For a robot the library already has, point at its USD and let the engine spawn it. On Isaac Sim, resolve the asset root and reference the file:

```
from isaacsim.storage.native import get_assets_root_path
usd = f"{get_assets_root_path()}/Isaac/Robots/Unitree/Go2/go2.usd"
```

On Isaac Lab, use the shipped config instead, which carries both the USD path and the tuned actuator groups:

```
from isaacsim.assets.robots.unitree import UNITREE_GO2_CFG
robot = Articulation(UNITREE_GO2_CFG.replace(prim_path="/World/robot"))
```

CHECKPOINT

The robot appears at its spawn height and holds a pose without folding. If it collapses, the drives carry placeholder gains, so continue to the bake in Step 4.

Step 4: Build a robot the shelf does not have

This is the Go2W path, and it applies to any Unitree model missing from the library. Unitree publishes official URDFs in `unitreerobotics/unitree_ros` under `robots/<model>_description`, with real masses, limits, and geometry. The pipeline is fetch, patch, import, bake, and it lives in `src/unitree_go2w_bake.py`:

Four things about the importer on this engine, all measured rather than documented:

- Importer 3.11.2 has no Kit commands. The documented `URDFParseAndImportFile` belongs to an older generation. Use `URDFImporter(URDFImporterConfig(...)).import_urdf()` from `isaacsim.asset.importer.urdf`.
- Its `usd_path` is a directory, not a filename. Output lands at `<usd_path>/<robot>/<robot>.usda` with payload files beside it, and the call returns the file it wrote.
- It converts primitive collisions and silently drops mesh collisions. A Go2W imported as-is has no wheel colliders at all and rests on its belly, so author convex hulls on the visual meshes yourself, per mesh rather than on the parent Xform.
- Patch `package://` mesh references to relative paths before importing.

The bake then writes gains into the USD DriveAPI so the file stands on its own. Two conversions matter: USD angular drives store stiffness and damping per degree while robot gain tables are per radian, so divide by 57.2958, and wheel joints take velocity drives with stiffness zero rather than position drives.

CHECKPOINT

The run reports the joint count it baked, the wheel colliders it authored, and a stand plus drive result. Verified reference: run 93cd6ff55fdd4742b0411d0f2514e803 stands at 0.335 m and drives 0.90 m in 2.5 s with a wheel radius of 0.086 m.

Step 5: Give it a controller

No pretrained locomotion policy ships with either engine for a wheeled quadruped, and Unitree's own RL repositories cover Go2, H1, and G1 rather than the W variants. The one public source of deployable checkpoints is `fan-ziqi/rl_sar`, which also deploys them to real robots over `unitree_sdk2`. It carries policies for `go2`, `go2w`, `b2`, `b2w`, `g1`, `a1`, `lite3`, `tita`, and others.

Read the contract from each policy's `config.yaml` rather than guessing it. The two we run differ in a way that matters:

	GO2W	G1
Joints	16, that is 12 legs and 4 wheels	29
Observation size	57	96
Legs	position targets, default + scale × action	position targets, all 29
Wheels	velocity targets, stiffness 0	none
Gains	legs kp 20 / kd 0.5, wheels kp 0 / kd 0.5	per joint, kp 20 to 200
Joint order	leg triplets FR, FL, RR, RL then wheels	base_joint_names[joint_mapping[i]]
Wheel positions in obs	zeroed, because the joints are continuous	not applicable

Both are velocity-commanded at the operator level, meaning you send a body twist of forward speed, lateral speed, and yaw rate at 50 Hz on a 200 Hz sim. Set the policy's own gains on the articulation controller at runtime, which is what the real deployment does.

CHECKPOINT

Verified reference runs, both on isaac-601-ga: b52b679e94db4b96aca4599b71b923a2 tracks a 0.5 m/s Go2W command at 96%, and 9dbf0a1627cc4a639f2127e1e8c090ba walks the G1 at 85% of a 0.4 m/s command with torso tilt under 5 degrees.

Step 6: Gate it on physics, not on video

A robot that looks fine in a clip can be sliding on frictionless wheels or holding a pose the drives cannot actually produce. Every scenario here asserts measured quantities, and the useful gates for legged robots are base height above a floor, tilt off vertical, joint tracking error, and real displacement over ground for a commanded velocity.

Allow a grace window at the start of each segment. A dropped-in robot rocks once while the controller catches it, and on the Go2W that transient reaches about 27 degrees of tilt before settling under 4 degrees. Judging the caught state rather than the transient is the difference between a meaningful gate and a flaky one.

Engine gotchas

SYMPTOM	CAUSE AND FIX
Every video films a frozen robot while physics clearly moves	The Fabric scene delegate severs USD from PhysX. Boot with <code>--/app/useFabricSceneDelegate=0,</code> <code>--/rtx/hydra/readTransformsFromFabricInRenderDelegate=0,</code> <code>--/physics/fabricEnabled=0,</code> and call <code>SimulationManager.enable_fabric(False)</code> after <code>world.reset()</code> , because a GPU boot turns it back on inside World.
On Isaac Lab, commands appear to be ignored	The Lab beta's default physics manager drops command writes. Pass <code>physics=OvPhysxCfg()</code> from <code>isaacclab_ovphysx</code> .
No module named 'isaacclab_assets'	You are on the Isaac Sim engine. Read the gain values from the upstream config source and author them yourself, or switch engines.
<code>import isaacsim.sensors</code> fails on Isaac Lab	That Kit extension is not enabled by default there. Use <code>isaacclab.sensors.Camera</code> , which needs <code>omni.replicator.core</code> and the carb flag <code>/isaacclab/cameras_enabled</code> .

SYMPTOM**CAUSE AND FIX****First render scenario on a fresh machine hangs**

The machine is cold. Warm it with a light render scenario first, roughly 25 seconds.

A camera returns black frames

The near clip plane is at its default. Set it explicitly, for example 0.05 m, and check that the subject fits the vertical field of view at your working distance.

The robot falls through the floor

Mesh collisions were dropped at import. Author convex hulls on the visual meshes of every collider-less link, including feet.

The Physical AI Sprint

A hackathon alongside Actuate SF

HOSTED BY



